



Allowed Functions Syntax Guide

47 authorized entries: declarations, parameters, returns, C++98 patterns, Webserv roles, and failure traps

Use this beside your implementation and manual pages. It emphasizes the subject's single-readiness-loop rule and separates portable APIs from Linux-only epoll and BSD/macOS kqueue.

How to read this guide

Each entry gives a practical declaration, required headers, parameter meanings, return convention, a small C++98 usage pattern, and the exact role the API normally plays in Webserv. The declarations may contain implementation-specific annotations on your machine, so the installed manual page remains the final authority for your evaluation host.

THE FOUR RULES TO MEMORIZE

Use one poll (or equivalent) for every socket and pipe, including listeners. Monitor reading and writing. Perform socket/pipe I/O only after matching readiness. Do not inspect `errno` after read/write to decide whether that speculative I/O was acceptable. Regular disk files are exempt from readiness monitoring.

Typical server lifecycle

resolve > socket > bind > listen > wait > accept > recv/send > close

Choose one readiness family

The subject allows poll or an equivalent. A normal implementation chooses one family for its central reactor; it does not combine all four merely because they are listed.

API	Where	Registration model	Main tradeoff
<code>select</code>	POSIX	Rebuild <code>fd_set</code> values each wait	Simple, but <code>FD_SETSIZE</code> and max-fd scanning limits
<code>poll</code>	POSIX	Provide a <code>pollfd</code> array each wait	Portable and explicit; linear array scan
<code>epoll</code>	Linux	Persistent kernel interest list	Scales well; platform-specific
<code>kqueue</code>	BSD/macOS	Submit changes; receive event records	Expressive; platform-specific

Allowed-list index

The subject names two API families as umbrellas: epoll expands to three functions and kqueue expands to two. errno is an error indicator rather than a callable function, but it is included because it appears in the authorized list.

Layer	Authorized entries
1. Error reporting and byte order	<code>errno</code> , <code>strerror</code> , <code>gai_strerror</code> , <code>htons</code> , <code>htonl</code> , <code>ntohs</code> , <code>ntohl</code>
2. Filesystem and descriptor I/O	<code>open</code> , <code>read</code> , <code>write</code> , <code>close</code> , <code>access</code> , <code>stat</code> , <code>chdir</code> , <code>opendir</code> , <code>readdir</code> , <code>closedir</code>
3. Processes, signals, and IPC	<code>fork</code> , <code>execve</code> , <code>pipe</code> , <code>socketpair</code> , <code>dup</code> , <code>dup2</code> , <code>waitpid</code> , <code>kill</code> , <code>signal</code>
4. Socket creation and networking	<code>socket</code> , <code>getaddrinfo</code> , <code>freeaddrinfo</code> , <code>getprotobyname</code> , <code>setsockopt</code> , <code>bind</code> , <code>listen</code> , <code>accept</code> , <code>connect</code> , <code>getsockname</code> , <code>send</code> , <code>recv</code> , <code>shutdown</code>
5. Readiness multiplexing	<code>select</code> , <code>poll</code> , <code>epoll_create</code> , <code>epoll_ctl</code> , <code>epoll_wait</code> , <code>kqueue</code> , <code>kevent</code>
6. Descriptor control	<code>fcntl</code>

MACOS FCNTL WARNING

Subject v24.1 permits only the stated fcntl forms involving F_SETFL, O_NONBLOCK, and FD_CLOEXEC. General POSIX recipes often add F_GETFL or F_SETFD; do not assume those extra commands are accepted by this evaluator.

Contents

1. Error reporting and byte order	6
1.1 errno	6
1.2 strerror	7
1.3 gai_strerror	8
1.4 htons	9
1.5 htonl	10
1.6 ntohs	11
1.7 ntohl	12
2. Filesystem and descriptor I/O	13
2.1 open	13
2.2 read	14
2.3 write	15
2.4 close	16
2.5 access	17
2.6 stat	18
2.7 chdir	19
2.8 opendir	20
2.9 readdir	21
2.10 closedir	22
3. Processes, signals, and IPC	23
3.1 fork	23
3.2 execve	24
3.3 pipe	25
3.4 socketpair	26
3.5 dup	27
3.6 dup2	28
3.7 waitpid	29
3.8 kill	30
3.9 signal	31
4. Socket creation and networking	32
4.1 socket	32
4.2 getaddrinfo	33
4.3 freeaddrinfo	34
4.4 getprotobyname	35
4.5 setsockopt	36
4.6 bind	37
4.7 listen	38
4.8 accept	39

4.9 connect	40
4.10 getsockname	41
4.11 send	42
4.12 recv	43
4.13 shutdown	44
5. Readiness multiplexing	45
5.1 select	45
5.2 poll	46
5.3 epoll_create	47
5.4 epoll_ctl	48
5.5 epoll_wait	49
5.6 kqueue	50
5.7 kevent	51
6. Descriptor control	52
6.1 fcntl	52
Reference spine	53

1. Error reporting and byte order

Interpret failures correctly and convert integer values between host and network byte order.

1.1 errno

KIND	Error indicator (not a function)	PLATFORM	C / POSIX
HEADERS	#include <cerrno>		

PROTOTYPE

```
// Conceptual form only; errno is normally a macro.  
int errno;
```

WHAT IT DOES

Provides the error code associated with a failed library or system call when that API documents errno reporting.

PARAMETERS

No parameters.

RETURN VALUE

There is no call and no return value. Read errno only after a function has reported failure. A successful call does not have to reset it.

ROLE IN WEBSERV

Useful when diagnosing failures from socket setup, open, fork, execve, poll, and similar calls. Save it immediately if another function might run before formatting the message.

C++98 USAGE PATTERN

```
int fd = socket(AF_INET, SOCK_STREAM, 0);  
if (fd == -1) {  
    int saved = errno;  
    std::cerr << "socket: " << std::strerror(saved) << std::endl;  
}
```

COMMON MISTAKES

- Never inspect errno unless the preceding API signalled failure.
- Save errno before calling logging or formatting code that may change it.
- The Webserv subject explicitly forbids checking errno after read or write to adjust server behaviour; readiness must drive those operations.

SUBJECT RULE

Special rule: do not use errno after read() or write() as the readiness/control-flow mechanism.

1.2 strerror

KIND	Error text helper	PLATFORM	C / POSIX
HEADERS	#include <cstring>		

PROTOTYPE

```
char *strerror(int errnum);
```

WHAT IT DOES

Returns a human-readable description for an errno-style numeric error code.

PARAMETERS

Parameter	Meaning
errnum	An error number, normally a saved copy of errno.

RETURN VALUE

A pointer to implementation-managed text. Copy it if it must survive later calls; do not free or modify it.

ROLE IN WEBSERV

Turns startup and runtime failures into useful diagnostics, for example bind: Address already in use.

C++98 USAGE PATTERN

```
int saved = errno;
std::cerr << "bind failed: "
           << std::strerror(saved) << std::endl;
```

COMMON MISTAKES

- Pass the saved error number, not an unrelated later errno value.
- The returned storage may be reused by later calls.
- getaddrinfo errors use gai_strerror, not strerror.

1.3 gai_strerror

KIND	Address-resolution error text helper	PLATFORM	POSIX sockets
HEADERS	#include <netdb.h>		

PROTOTYPE

```
const char *gai_strerror(int ecode);
```

WHAT IT DOES

Converts an EAI_* status returned by getaddrinfo into readable text.

PARAMETERS

Parameter	Meaning
ecode	The nonzero return code produced by getaddrinfo.

RETURN VALUE

A pointer to implementation-managed explanatory text. Do not free it.

ROLE IN WEBSERV

Reports configuration or listener address-resolution errors before startup.

C++98 USAGE PATTERN

```
int rc = getaddrinfo(host, service, &hints, &result);
if (rc != 0)
    std::cerr << "getaddrinfo: " << gai_strerror(rc) << std::endl;
```

COMMON MISTAKES

- getaddrinfo returns its own error code directly; it normally does not report failure as -1.
- Do not pass errno unless it happens to be the documented EAI code.

1.4 htons

KIND	Byte-order conversion	PLATFORM	POSIX sockets
HEADERS	#include <arpa/inet.h>		

PROTOTYPE

```
uint16_t htons(uint16_t hostshort);
```

WHAT IT DOES

Converts a 16-bit unsigned integer from host byte order to network byte order.

PARAMETERS

Parameter	Meaning
hostshort	A host-order 16-bit value, commonly a TCP or UDP port.

RETURN VALUE

The converted 16-bit value. There is no failure return.

ROLE IN WEBSERV

Stores a configured port in `sockaddr_in.sin_port` when building an IPv4 bind address manually.

C++98 USAGE PATTERN

```
struct sockaddr_in addr;  
std::memset(&addr, 0, sizeof(addr));  
addr.sin_family = AF_INET;  
addr.sin_port = htons(8080);
```

COMMON MISTAKES

- Use it for numeric 16-bit fields, not text such as "8080".
- Do not convert a value twice.

1.5 htonl

KIND	Byte-order conversion	PLATFORM	POSIX sockets
HEADERS	#include <arpa/inet.h>		

PROTOTYPE

```
uint32_t htonl(uint32_t hostlong);
```

WHAT IT DOES

Converts a 32-bit unsigned integer from host byte order to network byte order.

PARAMETERS

Parameter	Meaning
hostlong	A host-order 32-bit value.

RETURN VALUE

The converted 32-bit value. There is no failure return.

ROLE IN WEBSERV

Commonly used with IPv4 constants such as INADDR_ANY when constructing sockaddr_in manually.

C++98 USAGE PATTERN

```
struct sockaddr_in addr;  
std::memset(&addr, 0, sizeof(addr));  
addr.sin_family = AF_INET;  
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

COMMON MISTAKES

- The long in the name means a 32-bit network quantity, not necessarily C++ unsigned long on every ABI.
- Prefer getaddrinfo for general IPv4/IPv6 listener construction.

1.6 ntohs

KIND	Byte-order conversion	PLATFORM	POSIX sockets
HEADERS	#include <arpa/inet.h>		

PROTOTYPE

```
uint16_t ntohs(uint16_t netshort);
```

WHAT IT DOES

Converts a 16-bit unsigned integer from network byte order to host byte order.

PARAMETERS

Parameter	Meaning
netshort	A network-order 16-bit value, commonly <code>sin_port</code> .

RETURN VALUE

The converted host-order value. There is no failure return.

ROLE IN WEBSERV

Extracts a readable numeric port from a `sockaddr` returned by `getsockname` or `accept`.

C++98 USAGE PATTERN

```
unsigned int port = ntohs(addr.sin_port);  
std::cout << "listening on " << port << std::endl;
```

COMMON MISTAKES

- Apply it only to fields stored in network order.
- The returned integer can be printed or compared normally on the host.

1.7 ntohl

KIND	Byte-order conversion	PLATFORM	POSIX sockets
HEADERS	#include <arpa/inet.h>		

PROTOTYPE

```
uint32_t ntohl(uint32_t netlong);
```

WHAT IT DOES

Converts a 32-bit unsigned integer from network byte order to host byte order.

PARAMETERS

Parameter	Meaning
netlong	A network-order 32-bit value.

RETURN VALUE

The converted host-order value. There is no failure return.

ROLE IN WEBSERV

Useful when inspecting raw IPv4 numeric fields; most Webserv code should keep addresses inside sockaddr structures instead.

C++98 USAGE PATTERN

```
uint32_t host_value = ntohl(addr.sin_addr.s_addr);
```

COMMON MISTAKES

- This does not convert an address into printable dotted-decimal text.
- Do not assume IPv6 addresses fit into one 32-bit value.

2. Filesystem and descriptor I/O

Open static resources, read and write exact byte counts, inspect paths, and enumerate directories safely.

2.1 open

KIND	Open or create a filesystem object	PLATFORM	POSIX
HEADERS	#include <fcntl.h> #include <sys/types.h> #include <sys/stat.h>		

PROTOTYPE

```
int open(const char *path, int flags, ... /* mode_t mode */);
```

WHAT IT DOES

Opens a path and returns a descriptor. A third mode argument is required when flags create a file.

PARAMETERS

Parameter	Meaning
path	Null-terminated filesystem path.
flags	Access and behaviour flags such as O_RDONLY, O_WRONLY, O_CREAT, O_TRUNC, or O_APPEND.
mode	Permissions used for a newly created file, filtered by the process umask.

RETURN VALUE

A new nonnegative descriptor on success; -1 on failure with errno set.

ROLE IN WEBSERV

Opens static files, custom error pages, CGI scripts for validation, and upload destinations.

C++98 USAGE PATTERN

```
int fd = open(path.c_str(), O_RDONLY);  
if (fd == -1) {  
    // Build 403, 404, or 500 according to the real cause and policy.  
}
```

COMMON MISTAKES

- When O_CREAT is present, always supply mode, for example 0644.
- Validate and normalize the path before open; never concatenate an unchecked URL path.
- A previous access check does not guarantee open will succeed.

2.2 read

KIND	Descriptor input	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
ssize_t read(int fd, void *buf, size_t count);
```

WHAT IT DOES

Requests up to count bytes from a descriptor and writes them into buf.

PARAMETERS

Parameter	Meaning
fd	Readable descriptor.
buf	Valid writable memory for at least count bytes.
count	Maximum number of bytes requested.

RETURN VALUE

Positive byte count for progress; 0 for EOF; -1 for failure. A positive result may be smaller than count.

ROLE IN WEBSERV

Reads regular files and CGI pipes. Socket input can use read or recv, but all socket or pipe reads must follow readiness from the single multiplexer.

C++98 USAGE PATTERN

```
char buffer[4096];
ssize_t n = read(fd, buffer, sizeof(buffer));
if (n > 0)
    data.append(buffer, static_cast<size_t>(n));
```

COMMON MISTAKES

- count is a maximum, not a promised result.
- The buffer is not automatically null-terminated; use the returned length.
- For sockets and pipes, call only after read readiness. The subject forbids checking errno afterward to adjust behaviour.

SUBJECT RULE

Network and pipe reads must be non-blocking and preceded by readiness. Regular disk files are exempt.

2.3 write

KIND	Descriptor output	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
ssize_t write(int fd, const void *buf, size_t count);
```

WHAT IT DOES

Attempts to write up to count bytes from buf to a descriptor.

PARAMETERS

Parameter	Meaning
fd	Writable descriptor.
buf	Source memory containing at least count bytes.
count	Maximum number of bytes to write.

RETURN VALUE

Positive byte count for progress, possibly smaller than count; -1 on failure. A zero result means no bytes were written and must be handled deliberately.

ROLE IN WEBSERV

Writes upload files and CGI stdin. Socket responses can use write or send, with an output offset retained across partial writes.

C++98 USAGE PATTERN

```
ssize_t n = write(fd, data.data() + offset, data.size() - offset);
if (n > 0)
    offset += static_cast<size_t>(n);
```

COMMON MISTAKES

- Never assume one call writes the full buffer.
- Do not reuse or destroy the source bytes until the acknowledged prefix is accounted for.
- For sockets and pipes, call only after write readiness. The subject forbids checking errno afterward to adjust behaviour.

SUBJECT RULE

Network and pipe writes must be non-blocking and preceded by readiness. Regular disk files are exempt.

2.4 close

KIND	Release a descriptor	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int close(int fd);
```

WHAT IT DOES

Releases this process's reference to an open descriptor.

PARAMETERS

Parameter	Meaning
fd	An owned open descriptor.

RETURN VALUE

0 on success; -1 on failure with errno set.

ROLE IN WEBSERV

Closes listeners during shutdown, client sockets, files, CGI pipes, kqueue or epoll descriptors, and every unused pipe end after fork.

C++98 USAGE PATTERN

```
if (fd != -1) {  
    close(fd);  
    fd = -1;  
}
```

COMMON MISTAKES

- Erase event-loop registrations and ownership before the integer can be reused for another resource.
- Never close the same owned descriptor twice.
- After close, do not use fd even if close reported an error; platform semantics make blind retry unsafe.

2.5 access

KIND	Path accessibility check	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int access(const char *path, int mode);
```

WHAT IT DOES

Checks whether a path exists or appears accessible using requested permission tests.

PARAMETERS

Parameter	Meaning
path	Filesystem path.
mode	F_OK, or a combination of R_OK, W_OK, and X_OK.

RETURN VALUE

0 when all requested checks pass; -1 otherwise with errno set.

ROLE IN WEBSERV

Can validate configured CGI executables, roots, or upload directories, especially at startup.

C++98 USAGE PATTERN

```
if (access(script.c_str(), X_OK) == -1)
    throw ConfigError("CGI executable is not accessible");
```

COMMON MISTAKES

- It creates a time-of-check/time-of-use race; the later open, chdir, or execve result is authoritative.
- Do not use access as a security boundary for untrusted paths.

2.6 stat

KIND	Read filesystem metadata	PLATFORM	POSIX
HEADERS	#include <sys/types.h> #include <sys/stat.h> #include <unistd.h>		

PROTOTYPE

```
int stat(const char *path, struct stat *statbuf);
```

WHAT IT DOES

Retrieves metadata about the filesystem object named by path.

PARAMETERS

Parameter	Meaning
path	Filesystem path to inspect.
statbuf	Output structure receiving type, size, permissions, timestamps, and other metadata.

RETURN VALUE

0 on success; -1 on failure with errno set.

ROLE IN WEBSERV

Distinguishes regular files from directories, obtains static-file size, and validates route, upload, and CGI paths.

C++98 USAGE PATTERN

```
struct stat info;  
if (stat(path.c_str(), &info) == 0) {  
    if (S_ISREG(info.st_mode)) { /* file */ }  
    if (S_ISDIR(info.st_mode)) { /* directory */ }  
}
```

COMMON MISTAKES

- Use S_ISREG and S_ISDIR rather than comparing raw mode bits incorrectly.
- Metadata can change before a later open; handle the real operation's failure too.

2.7 chdir

KIND	Change process working directory	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int chdir(const char *path);
```

WHAT IT DOES

Changes the calling process's current working directory.

PARAMETERS

Parameter	Meaning
path	Directory that becomes the base for relative paths.

RETURN VALUE

0 on success; -1 on failure with errno set.

ROLE IN WEBSERV

Called in the CGI child before execve so script-relative files resolve from the correct directory.

C++98 USAGE PATTERN

```
if (child_pid == 0) {  
    if (chdir(script_dir.c_str()) == -1)  
        /* report child setup failure */;  
    execve(program, argv, envp);  
}
```

COMMON MISTAKES

- The current directory is process-wide. Do not chdir in the long-lived server parent for one request.
- Use the child after fork so other connections are unaffected.

2.8 opendir

KIND	Open a directory stream	PLATFORM	POSIX
HEADERS	#include <dirent.h>		

PROTOTYPE

```
DIR *opendir(const char *name);
```

WHAT IT DOES

Opens a directory for sequential entry enumeration.

PARAMETERS

Parameter	Meaning
name	Path of the directory to enumerate.

RETURN VALUE

A DIR pointer on success; NULL on failure with errno set.

ROLE IN WEBSERV

Begins generation of an autoindex page when a directory route permits listing.

C++98 USAGE PATTERN

```
DIR *dir = opendir(path.c_str());
if (dir == NULL)
    /* choose 403, 404, or 500 */;
```

COMMON MISTAKES

- DIR* is not an int descriptor and must be released with closedir.
- Escape display names and URL-encode links generated from entries.

2.9 readdir

KIND	Read one directory entry	PLATFORM	POSIX
HEADERS	#include <dirent.h>		

PROTOTYPE

```
struct dirent *readdir(DIR *dirp);
```

WHAT IT DOES

Returns the next entry from an open directory stream.

PARAMETERS

Parameter	Meaning
dirp	A valid DIR pointer returned by opendir.

RETURN VALUE

A pointer to the next dirent, or NULL at end-of-directory or on error. The returned storage is managed by the directory stream.

ROLE IN WEBSERV

Enumerates names used to construct a directory-listing response.

C++98 USAGE PATTERN

```
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    std::string name(entry->d_name);
    // Filter, escape, and encode before generating HTML.
}
```

COMMON MISTAKES

- The returned pointer can be overwritten by the next call; copy needed fields.
- Skip or handle . and .. deliberately.
- To distinguish EOF from error, set errno to 0 before the loop and inspect it only after NULL; this is not a read() call.

2.10 closedir

KIND	Close a directory stream	PLATFORM	POSIX
HEADERS	#include <dirent.h>		

PROTOTYPE

```
int closedir(DIR *dirp);
```

WHAT IT DOES

Closes a DIR stream and its underlying resources.

PARAMETERS

Parameter	Meaning
dirp	An open directory stream.

RETURN VALUE

0 on success; -1 on failure with errno set.

ROLE IN WEBSERV

Completes autoindex enumeration on both success and every error path.

C++98 USAGE PATTERN

```
if (dir != NULL) {
    closedir(dir);
    dir = NULL;
}
```

COMMON MISTAKES

- Do not call close on the internal descriptor instead of closedir.
- Every successful opendir needs exactly one closedir.

3. Processes, signals, and IPC

Create CGI children, connect their standard streams, enforce timeouts, and collect their termination status.

3.1 fork

KIND	Create a child process	PLATFORM	POSIX
HEADERS	#include <sys/types.h> #include <unistd.h>		

PROTOTYPE

```
pid_t fork(void);
```

WHAT IT DOES

Creates a child process that initially shares copies of the parent's process state and references to the same open file descriptions.

PARAMETERS

No parameters.

RETURN VALUE

-1 in the parent on failure; 0 in the child; the child's positive PID in the parent on success.

ROLE IN WEBSERV

Creates a process only for a configured CGI request. The child prepares stdin, stdout, environment, and working directory before `execve`.

C++98 USAGE PATTERN

```
pid_t pid = fork();
if (pid == -1) {
    /* CGI creation failed */
} else if (pid == 0) {
    /* child: dup2, close, chdir, execve */
} else {
    /* parent: store pid and monitor pipes */
}
```

COMMON MISTAKES

- Both processes continue at the next instruction; branch on the return value immediately.
- Close every pipe end that each process does not use, or EOF may never arrive.
- Do not duplicate buffered output accidentally in both processes.

SUBJECT RULE

`fork()` is forbidden for client concurrency and may be used only for CGI.

3.2 execve

KIND	Replace a process image	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

WHAT IT DOES

Replaces the current process with the executable at pathname, using explicit argument and environment vectors.

PARAMETERS

Parameter	Meaning
pathname	Executable file to load, such as a Python interpreter.
argv	NULL-terminated argument-pointer array; argv[0] conventionally names the program.
envp	NULL-terminated environment-pointer array containing CGI variables.

RETURN VALUE

It does not return on success. It returns -1 only when replacement fails, with errno set.

ROLE IN WEBSERV

Runs the CGI interpreter or executable after the child connects its pipes, changes directory, and constructs validated CGI environment variables.

C++98 USAGE PATTERN

```
char *argv[] = { program, script, NULL };
char *envp[] = { method_env, query_env, NULL };
execve(program, argv, envp);
// Reached only on failure.
/* report setup failure and terminate child */
```

COMMON MISTAKES

- Do not pass pointers to strings whose storage has already expired.
- argv and envp arrays, and every string they reference, must remain valid until the call.
- No shell performs quoting, PATH lookup, redirection, or wildcard expansion for you.

SUBJECT RULE

Never execve another web server. Use it only to execute configured CGI processing.

3.3 pipe

KIND	Create a one-way byte stream	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int pipe(int pipefd[2]);
```

WHAT IT DOES

Creates a unidirectional kernel pipe with a read end and a write end.

PARAMETERS

Parameter	Meaning
pipefd	Two-element output array: pipefd[0] is readable and pipefd[1] is writable.

RETURN VALUE

0 on success; -1 on failure. On success both returned descriptors must be owned and closed correctly.

ROLE IN WEBSERV

Two pipes normally connect Webserv to CGI: server-to-child stdin and child-to-server stdout.

C++98 USAGE PATTERN

```
int to_child[2];
int from_child[2];
if (pipe(to_child) == -1 || pipe(from_child) == -1)
    /* close anything created and fail CGI setup */;
```

COMMON MISTAKES

- A pipe is one-way; full-duplex CGI normally needs two pipes.
- Pipe buffers are finite. Monitor both directions concurrently to avoid deadlock.
- Set the parent ends non-blocking and register them in the single readiness multiplexer.

SUBJECT RULE

CGI pipe I/O can wait, so it must be non-blocking and readiness-driven by the same central multiplexer.

3.4 socketpair

KIND	Create a connected local socket pair	PLATFORM	POSIX / Unix
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int socketpair(int domain, int type, int protocol, int sv[2]);
```

WHAT IT DOES

Creates two connected, usually full-duplex, local sockets.

PARAMETERS

Parameter	Meaning
domain	Normally AF_UNIX for local interprocess communication.
type	Usually SOCK_STREAM for a byte stream.
protocol	Normally 0 to choose the default protocol.
sv	Two-element output array containing the connected descriptors.

RETURN VALUE

0 on success; -1 on failure with errno set.

ROLE IN WEBSERV

Can provide bidirectional parent-child CGI communication as an alternative to two one-way pipes, if the team's design and evaluation support it.

C++98 USAGE PATTERN

```
int sv[2];
if (socketpair(AF_UNIX, SOCK_STREAM, 0, sv) == -1)
    /* CGI IPC setup failed */;
```

COMMON MISTAKES

- Close the unused endpoint in each process after fork.
- It is still stream I/O: reads and writes can be partial and must be readiness-driven.
- Do not confuse local socketpair endpoints with client TCP sockets.

3.5 dup

KIND	Duplicate a descriptor	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int dup(int oldfd);
```

WHAT IT DOES

Returns a new descriptor referring to the same open file description as oldfd.

PARAMETERS

Parameter	Meaning
oldfd	An existing valid descriptor.

RETURN VALUE

The lowest available nonnegative descriptor on success; -1 on failure.

ROLE IN WEBSERV

Can preserve or duplicate an existing descriptor during CGI setup, though dup2 is normally more direct for stdin and stdout.

C++98 USAGE PATTERN

```
int copy = dup(original_fd);
if (copy == -1)
    /* duplication failed */;
```

COMMON MISTAKES

- Both descriptors share the same underlying file offset and status flags.
- Each descriptor number must eventually be closed once.
- dup does not let you choose the new descriptor number.

3.6 dup2

KIND	Duplicate onto a chosen descriptor	PLATFORM	POSIX
HEADERS	#include <unistd.h>		

PROTOTYPE

```
int dup2(int oldfd, int newfd);
```

WHAT IT DOES

Makes newfd refer to the same open file description as oldfd, closing a previously open newfd atomically when necessary.

PARAMETERS

Parameter	Meaning
oldfd	Source descriptor.
newfd	Exact descriptor number that should become the duplicate.

RETURN VALUE

newfd on success; -1 on failure.

ROLE IN WEBSERV

Connects a CGI child's input pipe to STDIN_FILENO and its output pipe to STDOUT_FILENO before execve.

C++98 USAGE PATTERN

```
if (dup2(to_child[0], STDIN_FILENO) == -1)
    /* child setup failed */;
if (dup2(from_child[1], STDOUT_FILENO) == -1)
    /* child setup failed */;
```

COMMON MISTAKES

- After successful duplication, close the original pipe descriptors that the child no longer needs.
- If oldfd equals newfd and oldfd is valid, dup2 returns newfd without changing it.

3.7 waitpid

KIND	Collect child state	PLATFORM	POSIX
HEADERS	#include <sys/types.h> #include <sys/wait.h>		

PROTOTYPE

```
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

WHAT IT DOES

Reports and collects a selected child's state change, preventing exited children from remaining zombies.

PARAMETERS

Parameter	Meaning
pid	Child selection; a positive value waits for that exact PID.
wstatus	Optional output status interpreted with WIFEXITED, WEXITSTATUS, WIFSIGNALED, and related macros.
options	Use WNOHANG in the event loop to avoid blocking.

RETURN VALUE

The child PID when collected; 0 with WNOHANG when no selected child has changed; -1 on failure.

ROLE IN WEBSERV

Checks whether each CGI has exited after pipe events or during periodic cleanup and timeout processing.

C++98 USAGE PATTERN

```
int status = 0;
pid_t rc = waitpid(cgi_pid, &status, WNOHANG);
if (rc == cgi_pid && WIFEXITED(status))
    exit_code = WEXITSTATUS(status);
```

COMMON MISTAKES

- Calling without WNOHANG can freeze every client while CGI is still running.
- Pipe EOF and child exit are related but distinct events; track both.
- Every created CGI child must eventually be reaped.

3.8 kill

KIND	Send a signal	PLATFORM	POSIX
HEADERS	#include <sys/types.h> #include <signal.h>		

PROTOTYPE

```
int kill(pid_t pid, int sig);
```

WHAT IT DOES

Sends a signal to a process or process group selected by pid.

PARAMETERS

Parameter	Meaning
pid	Target PID; use the known positive CGI child PID for precise control.
sig	Signal such as SIGTERM or SIGKILL; signal 0 performs an existence/permission check without delivering a signal.

RETURN VALUE

0 when the signal was successfully sent; -1 on failure.

ROLE IN WEBSERV

Stops a CGI that exceeded its deadline or whose client disconnected, followed by waitpid to reap it.

C++98 USAGE PATTERN

```
if (kill(cgi_pid, SIGTERM) == -1)
    /* record failure, but still attempt cleanup and waitpid */;
```

COMMON MISTAKES

- A successful return means the signal was sent, not that the process has already exited.
- Never call kill with an unvalidated or stale PID.
- Use a bounded escalation policy and always reap the child.

3.9 signal

KIND	Install a simple signal disposition	PLATFORM	C / POSIX
HEADERS	#include <signal.h>		

PROTOTYPE

```
void (*signal(int sig, void (*handler)(int)))(int);
```

WHAT IT DOES

Sets a signal's disposition to a handler function, SIG_IGN, or SIG_DFL.

PARAMETERS

Parameter	Meaning
sig	Signal number to configure.
handler	Handler address, SIG_IGN to ignore, or SIG_DFL for default behaviour.

RETURN VALUE

The previous disposition on success; SIG_ERR on failure.

ROLE IN WEBSERV

Commonly ignores SIGPIPE so sending to a disconnected client reports a write failure instead of terminating the whole server.

C++98 USAGE PATTERN

```
if (signal(SIGPIPE, SIG_IGN) == SIG_ERR)
    /* startup signal configuration failed */;
```

COMMON MISTAKES

- signal has historical platform-specific semantics; keep its use minimal and simple because it is the authorized interface.
- Do not perform complex non-async-signal-safe work inside a handler.
- Ignoring SIGPIPE does not remove the need to handle failed socket writes and clean the connection.

4. Socket creation and networking

Resolve listener addresses, create and configure sockets, accept clients, and exchange bytes without confusing TCP with HTTP message boundaries.

4.1 socket

KIND	Create a communication endpoint	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int socket(int domain, int type, int protocol);
```

WHAT IT DOES

Creates an unbound socket endpoint and returns its descriptor.

PARAMETERS

Parameter	Meaning
domain	Address family, commonly AF_INET or AF_INET6.
type	Communication semantics; SOCK_STREAM creates a TCP-style byte stream.
protocol	Usually 0 to select the default protocol for the family and type.

RETURN VALUE

A nonnegative socket descriptor on success; -1 on failure.

ROLE IN WEBSERV

Creates every listening socket before setsockopt, bind, and listen.

C++98 USAGE PATTERN

```
int fd = socket(address->ai_family,  
               address->ai_socktype,  
               address->ai_protocol);  
if (fd == -1)  
    /* try next resolved address or fail startup */;
```

COMMON MISTAKES

- A newly created socket is not yet bound, listening, accepted, or connected.
- Close it on every later setup failure.
- Set the non-blocking behaviour required by the subject before placing it in the event loop.

4.2 getaddrinfo

KIND	Resolve host and service into socket addresses	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h> #include <netdb.h>		

PROTOTYPE

```
int getaddrinfo(const char *node, const char *service,  
               const struct addrinfo *hints,  
               struct addrinfo **res);
```

WHAT IT DOES

Builds a linked list of addresses matching host, service, family, socket type, protocol, and flags.

PARAMETERS

Parameter	Meaning
node	Host or interface name; NULL with AI_PASSIVE requests wildcard listener addresses.
service	Service name or decimal port string.
hints	Optional filters such as AF_UNSPEC, SOCK_STREAM, and AI_PASSIVE.
res	Output pointer receiving a linked list that must be freed with freeaddrinfo.

RETURN VALUE

0 on success; a nonzero EAI_* code on failure. Use gai_strerror for that code, not errno-style -1 handling.

ROLE IN WEBSERV

Converts every configured interface and port into candidate sockaddr records for socket and bind, supporting IPv4 and IPv6 without manual layout assumptions.

C++98 USAGE PATTERN

```
struct addrinfo hints;  
std::memset(&hints, 0, sizeof(hints));  
hints.ai_family = AF_UNSPEC;  
hints.ai_socktype = SOCK_STREAM;  
hints.ai_flags = AI_PASSIVE;  
struct addrinfo *result = NULL;  
int rc = getaddrinfo(NULL, "8080", &hints, &result);
```

COMMON MISTAKES

- Iterate candidates because the first address may fail to create or bind.
- Do not copy only part of a sockaddr based on guessed family size.
- Always free a successful result list.

4.3 freeaddrinfo

KIND	Release an address list	PLATFORM	POSIX sockets
HEADERS	#include <netdb.h>		

PROTOTYPE

```
void freeaddrinfo(struct addrinfo *res);
```

WHAT IT DOES

Frees the linked list allocated by a successful getaddrinfo call.

PARAMETERS

Parameter	Meaning
res	The head pointer returned through getaddrinfo.

RETURN VALUE

No return value.

ROLE IN WEBSERV

Releases temporary address-resolution data after creating listener sockets or abandoning startup.

C++98 USAGE PATTERN

```
if (result != NULL) {  
    freeaddrinfo(result);  
    result = NULL;  
}
```

COMMON MISTAKES

- Do not free individual ai_addr members separately.
- Do not use any addrinfo pointer after the list is freed.

4.4 getprotobyname

KIND	Look up a protocol database entry	PLATFORM	POSIX sockets
HEADERS	#include <netdb.h>		

PROTOTYPE

```
struct protoent *getprotobyname(const char *name);
```

WHAT IT DOES

Returns protocol database information for a name such as tcp.

PARAMETERS

Parameter	Meaning
name	Null-terminated protocol name.

RETURN VALUE

A pointer to implementation-managed struct protoent on success; NULL when no entry is found or an error occurs.

ROLE IN WEBSERV

Can obtain the numeric protocol value for TCP, although socket(..., SOCK_STREAM, 0) normally selects the default protocol directly.

C++98 USAGE PATTERN

```
struct protoent *tcp = getprotobyname("tcp");
if (tcp == NULL)
    /* protocol database lookup failed */;
```

COMMON MISTAKES

- The returned storage may be overwritten by later network-database calls; copy needed fields.
- This resolves protocol names, not hostnames or ports.

4.5 setsockopt

KIND	Configure a socket option	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int setsockopt(int socket, int level, int option_name,  
              const void *option_value, socklen_t option_len);
```

WHAT IT DOES

Sets one option on an existing socket at a specified protocol level.

PARAMETERS

Parameter	Meaning
socket	Socket descriptor to configure.
level	Option level, commonly SOL_SOCKET.
option_name	Specific option such as SO_REUSEADDR.
option_value	Pointer to the option value.
option_len	Byte size of the supplied value.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Sets SO_REUSEADDR before bind so a listener can restart without unnecessary address-in-use delays.

C++98 USAGE PATTERN

```
int enabled = 1;  
if (setsockopt(fd, SOL_SOCKET, SO_REUSEADDR,  
              &enabled, sizeof(enabled)) == -1)  
    /* close fd and fail or try another address */;
```

COMMON MISTAKES

- Set options at the correct level and before the operation they affect.
- SO_REUSEADDR does not make arbitrary duplicate active listeners safe.
- The option value type and size are option-specific.

4.6 bind

KIND	Assign a local socket address	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int bind(int socket, const struct sockaddr *address,  
        socklen_t address_len);
```

WHAT IT DOES

Associates a socket with a local interface address and port.

PARAMETERS

Parameter	Meaning
socket	Unbound socket descriptor.
address	Pointer to a family-specific address cast to sockaddr.
address_len	Exact byte size of that address structure.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Applies each configured interface-and-port pair to a listener candidate before listen.

C++98 USAGE PATTERN

```
if (bind(fd, address->ai_addr, address->ai_addrlen) == -1) {  
    close(fd);  
    /* try next candidate or report startup failure */  
}
```

COMMON MISTAKES

- Use the correct address length for its family.
- Common failures include an occupied port, unavailable interface, or insufficient privilege.
- A successful bind does not yet make the socket accept connections.

4.7 listen

KIND	Mark a stream socket as passive	PLATFORM	POSIX sockets
HEADERS	#include <sys/socket.h>		

PROTOTYPE

```
int listen(int socket, int backlog);
```

WHAT IT DOES

Turns a bound SOCK_STREAM socket into a listening socket for incoming connections.

PARAMETERS

Parameter	Meaning
socket	A successfully bound stream socket.
backlog	Requested limit related to pending connections; the kernel may cap it.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Finalizes every configured listener before registering it for read readiness in the central event loop.

C++98 USAGE PATTERN

```
if (listen(fd, SOMAXCONN) == -1) {  
    close(fd);  
    /* listener setup failed */  
}
```

COMMON MISTAKES

- The backlog is not the total number of clients Webserv can ever serve.
- The listening descriptor never carries an individual client's HTTP bytes.

4.8 accept

KIND	Create a socket for one incoming client	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int accept(int socket, struct sockaddr *address,  
           socklen_t *address_len);
```

WHAT IT DOES

Removes one pending connection from a listening socket and returns a new connected socket descriptor.

PARAMETERS

Parameter	Meaning
socket	Listening socket that reported read readiness.
address	Optional output buffer for the peer address; may be NULL.
address_len	In/out size pointer for address; may be NULL when address is NULL.

RETURN VALUE

A new nonnegative client descriptor on success; -1 on failure.

ROLE IN WEBSERV

Creates one Connection object per accepted client while leaving the listening descriptor open for future clients.

C++98 USAGE PATTERN

```
struct sockaddr_storage peer;  
socklen_t length = sizeof(peer);  
int client = accept(listener,  
                    reinterpret_cast<struct sockaddr *>(&peer),  
                    &length);
```

COMMON MISTAKES

- Call only after listener readiness under the subject's one-multiplexer rule.
- On Linux the accepted socket does not necessarily inherit O_NONBLOCK; configure it explicitly.
- The new descriptor and listener have different ownership and roles.

SUBJECT RULE

The listening socket and all accepted sockets belong to the same central readiness system.

4.9 connect

KIND	Initiate a connection	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int connect(int socket, const struct sockaddr *address,  
            socklen_t address_len);
```

WHAT IT DOES

Initiates a connection from a socket to a peer address.

PARAMETERS

Parameter	Meaning
socket	Socket that will become connected.
address	Destination address.
address_len	Byte size of the destination structure.

RETURN VALUE

0 when connected immediately; -1 on failure or when a non-blocking connection is still in progress, depending on documented error state.

ROLE IN WEBSERV

A basic HTTP server normally accepts inbound clients and may never need connect. It is available for designs requiring an outbound socket, but not for delegating to another web server.

C++98 USAGE PATTERN

```
int rc = connect(fd, address->ai_addr, address->ai_addrlen);  
if (rc == -1) {  
    /* general non-blocking clients normally monitor completion */  
}
```

COMMON MISTAKES

- A non-blocking connect is a multi-step operation and requires platform-correct completion checks.
- Do not use it to proxy the assignment to another web server.
- Most Webserv implementations should not invent an outbound dependency they do not need.

4.10 getsockname

KIND	Read a socket's local address	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
int getsockname(int socket, struct sockaddr *address,  
                socklen_t *address_len);
```

WHAT IT DOES

Retrieves the local address currently assigned to a socket.

PARAMETERS

Parameter	Meaning
socket	Socket descriptor to inspect.
address	Output storage for the local address.
address_len	In/out size: capacity on entry, actual size on success.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Can discover the actual port after binding port 0 or verify which local endpoint accepted a connection.

C++98 USAGE PATTERN

```
struct sockaddr_storage local;  
socklen_t length = sizeof(local);  
if (getsockname(fd,  
                reinterpret_cast<struct sockaddr *>(&local), &length) == -1)  
    /* inspection failed */;
```

COMMON MISTAKES

- Initialize address_len before the call.
- Interpret the returned storage according to its sa_family and use ntohs for network-order ports.

4.11 send

KIND	Send bytes through a socket	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
ssize_t send(int socket, const void *buffer,  
             size_t length, int flags);
```

WHAT IT DOES

Attempts to transmit up to length bytes from buffer through a connected socket.

PARAMETERS

Parameter	Meaning
socket	Connected socket descriptor.
buffer	Source bytes.
length	Maximum bytes requested.
flags	Per-call flags; 0 is the basic Webserv case.

RETURN VALUE

Positive number of bytes accepted, possibly short; -1 on failure. Zero means no progress and needs a defined policy.

ROLE IN WEBSERV

Sends queued HTTP response bytes after the client socket reports write readiness.

C++98 USAGE PATTERN

```
ssize_t n = send(client, output.data() + offset,  
                 output.size() - offset, 0);  
if (n > 0)  
    offset += static_cast<size_t>(n);
```

COMMON MISTAKES

- One call does not equal one complete HTTP response.
- Keep the unsent suffix and request write readiness again.
- Handle SIGPIPE so a disconnected peer cannot terminate Webserv.

SUBJECT RULE

Call only after write readiness. Do not use errno after the call to turn speculative writes into the event mechanism.

4.12 recv

KIND	Receive bytes from a socket	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/socket.h>		

PROTOTYPE

```
ssize_t recv(int socket, void *buffer, size_t length, int flags);
```

WHAT IT DOES

Receives up to length bytes from a connected socket into buffer.

PARAMETERS

Parameter	Meaning
socket	Connected client socket.
buffer	Writable destination memory.
length	Maximum byte count.
flags	Per-call flags; 0 is the basic Webserv case.

RETURN VALUE

Positive byte count for progress; 0 for orderly peer shutdown of the sending direction; -1 on failure.

ROLE IN WEBSERV

Adds available TCP bytes to the Connection input buffer after read readiness, then resumes the incremental HTTP parser.

C++98 USAGE PATTERN

```
char buffer[8192];
ssize_t n = recv(client, buffer, sizeof(buffer), 0);
if (n > 0)
    input.append(buffer, static_cast<size_t>(n));
else if (n == 0)
    /* peer closed its sending side */;
```

COMMON MISTAKES

- TCP has no HTTP message boundaries: one recv may contain a fragment or several requests.
- Do not null-terminate and use strlen for binary-safe bodies.
- A positive short result is not EOF.

SUBJECT RULE

Call only after read readiness. Do not use errno after the call to turn speculative reads into the event mechanism.

4.13 shutdown

KIND	Disable socket communication directions	PLATFORM	POSIX sockets
HEADERS	#include <sys/socket.h>		

PROTOTYPE

```
int shutdown(int socket, int how);
```

WHAT IT DOES

Disables further reception, transmission, or both on a connected socket while the descriptor remains open.

PARAMETERS

Parameter	Meaning
socket	Connected socket descriptor.
how	SHUT_RD, SHUT_WR, or SHUT_RDWR.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Can make a deliberate half-close, for example signalling that no more response bytes will be sent before final close, although many connection cleanup paths simply close.

C++98 USAGE PATTERN

```
if (shutdown(client, SHUT_WR) == -1)
    /* peer may already be gone; continue controlled cleanup */;
```

COMMON MISTAKES

- shutdown does not release the descriptor; close is still required.
- It affects the socket communication state, including duplicated descriptors referring to that socket.
- Do not confuse SHUT_WR with closing only one integer descriptor copy.

5. Readiness multiplexing

Wait once, then perform socket or pipe I/O only for descriptors that reported the required readiness.

5.1 select

KIND	Descriptor readiness multiplexer	PLATFORM	POSIX sockets
HEADERS	#include <sys/types.h> #include <sys/time.h> #include <sys/select.h>		

PROTOTYPE

```
int select(int nfds, fd_set *readfds, fd_set *writefds,  
          fd_set *exceptfds, struct timeval *timeout);
```

WHAT IT DOES

Waits until one or more descriptors become ready, a timeout expires, or the wait is interrupted.

PARAMETERS

Parameter	Meaning
<code>nfds</code>	One more than the highest-numbered descriptor present in any set.
<code>readfds</code>	Descriptors to test for read readiness; may be NULL.
<code>writefds</code>	Descriptors to test for write readiness; may be NULL.
<code>exceptfds</code>	Descriptors to test for exceptional conditions; may be NULL.
<code>timeout</code>	Maximum wait; NULL blocks indefinitely and a zero timeval polls immediately.

RETURN VALUE

Positive number of ready descriptor bits; 0 on timeout; -1 on failure. The descriptor sets are modified to contain only ready descriptors.

ROLE IN WEBSERV

Can be the server's single event loop: watch listeners and clients for reading, and watch clients or CGI pipes for writing only while output is pending.

C++98 USAGE PATTERN

```
fd_set readable;  
FD_ZERO(&readable); FD_SET(listener, &readable);  
int n = select(listener + 1, &readable, NULL, NULL, NULL);  
if (n > 0 && FD_ISSET(listener, &readable))  
    /* accept is now permitted */;
```

COMMON MISTAKES

- Rebuild every `fd_set` because `select` overwrites the sets.
- Reinitialize timeout because some systems modify it.
- Every descriptor must be below `FD_SETSIZE`; `select` scales poorly to large sets.
- `nfds` is `max_fd + 1`, not the descriptor count.

SUBJECT RULE

If chosen, `select` is the one central readiness mechanism. Socket or pipe I/O requires the corresponding returned readiness bit.

5.2 poll

KIND	Descriptor readiness multiplexer	PLATFORM	POSIX
HEADERS	#include <poll.h>		

PROTOTYPE

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);

struct pollfd {
    int fd;
    short events;
    short revents;
};
```

WHAT IT DOES

Waits for requested events on an array of descriptors without select's numeric FD_SETSIZE limit.

PARAMETERS

Parameter	Meaning
fds	Array whose events fields request conditions and whose revents fields receive results.
nfds	Number of pollfd elements in the array.
timeout	Milliseconds to wait: -1 indefinitely, 0 immediately, or a positive deadline.

RETURN VALUE

Positive count of array elements with nonzero revents; 0 on timeout; -1 on failure.

ROLE IN WEBSERV

A natural portable central loop. Request POLLIN for listeners, client input, and CGI output; request POLLOUT only where queued bytes need to be flushed.

C++98 USAGE PATTERN

```
struct pollfd item;
item.fd = listener;
item.events = POLLIN;
item.revents = 0;
int ready = poll(&item, 1, 1000);
if (ready > 0 && (item.revents & POLLIN))
    /* accept is now permitted */;
```

COMMON MISTAKES

- Inspect revents, not events, after poll returns.
- Always handle POLLERR, POLLHUP, and POLLNVAL; they may appear even when not requested.
- Continuously requesting POLLOUT causes busy loops because sockets are often writable.
- The array is scanned linearly, so keep descriptor bookkeeping disciplined.

SUBJECT RULE

A single poll call (or an equivalent API) must monitor all socket and CGI-pipe I/O, including listening sockets. Read or write only after the matching readiness result.

5.3 epoll_create

KIND	Create an epoll instance	PLATFORM	Linux only
HEADERS	#include <sys/epoll.h>		

PROTOTYPE

```
int epoll_create(int size);
```

WHAT IT DOES

Creates a kernel epoll interest-list instance and returns the descriptor used by `epoll_ctl` and `epoll_wait`.

PARAMETERS

Parameter	Meaning
size	Must be greater than zero; modern Linux ignores the value as a sizing hint.

RETURN VALUE

A new epoll descriptor on success; -1 on failure.

ROLE IN WEBSERV

Creates the one Linux event-loop object used to monitor listeners, clients, and CGI pipe ends.

C++98 USAGE PATTERN

```
int epfd = epoll_create(64);
if (epfd == -1)
    /* startup failed */;
/* close(epfd) during shutdown */
```

COMMON MISTAKES

- The returned epoll descriptor must be closed.
- epoll is Linux-specific; it is not available on macOS or BSD.
- The subject names `epoll_create`, so do not silently substitute an unlisted convenience API.

SUBJECT RULE

Use this only when epoll is the selected equivalent of the mandatory central poll mechanism.

5.4 epoll_ctl

KIND	Manage an epoll interest list	PLATFORM	Linux only
HEADERS	#include <sys/epoll.h>		

PROTOTYPE

```
int epoll_ctl(int epfd, int op, int fd,
              struct epoll_event *event);
```

WHAT IT DOES

Adds, modifies, or removes a descriptor registration in an epoll instance.

PARAMETERS

Parameter	Meaning
epfd	Descriptor returned by epoll_create.
op	EPOLL_CTL_ADD, EPOLL_CTL_MOD, or EPOLL_CTL_DEL.
fd	Descriptor whose registration is being changed.
event	Requested events and user data for ADD/MOD; follow host documentation for DEL.

RETURN VALUE

0 on success; -1 on failure.

ROLE IN WEBSERV

Registers EPOLLIN for inbound work and enables EPOLLOUT only while a connection or pipe has queued output.

C++98 USAGE PATTERN

```
struct epoll_event event;
std::memset(&event, 0, sizeof(event));
event.events = EPOLLIN;
event.data.fd = client;
if (epoll_ctl(epfd, EPOLL_CTL_ADD, client, &event) == -1)
    /* registration failed */;
```

COMMON MISTAKES

- ADD fails if the descriptor is already registered; use MOD for an existing registration.
- Remove stale registrations and close descriptors through one controlled cleanup path.
- Treat EPOLLERR and EPOLLHUP as lifecycle events even if only EPOLLIN/EPOLLOUT were requested.
- Edge-triggered EPOLLET requires a carefully designed drain-until-blocked loop; level-triggered mode is simpler for this subject.

SUBJECT RULE

All monitored socket and pipe descriptors must participate in the same epoll instance if epoll is your chosen equivalent.

5.5 epoll_wait

KIND	Wait for epoll events	PLATFORM	Linux only
HEADERS	#include <sys/epoll.h>		

PROTOTYPE

```
int epoll_wait(int epfd, struct epoll_event *events,
               int maxevents, int timeout);
```

WHAT IT DOES

Waits for ready events from an epoll instance and writes them into a caller-provided array.

PARAMETERS

Parameter	Meaning
epfd	Descriptor returned by epoll_create.
events	Output array for ready event records.
maxevents	Positive capacity of the output array.
timeout	Milliseconds: -1 indefinitely, 0 immediately, or a positive deadline.

RETURN VALUE

Positive number of initialized events; 0 on timeout; -1 on failure.

ROLE IN WEBSERV

Drives a Linux event loop by dispatching only the events in events[0..ready-1], then updating registrations as connection state changes.

C++98 USAGE PATTERN

```
struct epoll_event events[64];
int ready = epoll_wait(epfd, events, 64, 1000);
for (int i = 0; i < ready; ++i) {
    if (events[i].events & EPOLLIN)
        /* perform the permitted read-side operation */;
}
```

COMMON MISTAKES

- Only the first return-value elements are valid.
- A finite timeout is useful when the server must enforce request or CGI deadlines.
- Do not assume every registered descriptor appears on each return.
- Handle interruption according to a deliberate signal policy, not as an arbitrary fatal event.

SUBJECT RULE

The returned event flags are the permission to perform socket or pipe I/O; do not probe descriptors outside this result set.

5.6 kqueue

KIND	Create a kernel event queue	PLATFORM	BSD and macOS
HEADERS	#include <sys/types.h> #include <sys/event.h> #include <sys/time.h>		

PROTOTYPE

```
int kqueue(void);
```

WHAT IT DOES

Creates a kernel event queue used with kevent to register interests and receive readiness notifications.

PARAMETERS

No parameters.

RETURN VALUE

A new queue descriptor on success; -1 on failure.

ROLE IN WEBSERV

Creates the one BSD/macOS event-loop object used for listeners, clients, and CGI pipe ends.

C++98 USAGE PATTERN

```
int queue_fd = kqueue();  
if (queue_fd == -1)  
    /* startup failed */;  
/* close(queue_fd) during shutdown */
```

COMMON MISTAKES

- The returned queue descriptor must be closed.
- kqueue is not a Linux API.
- Create one central queue instead of a separate queue per connection.

SUBJECT RULE

Use this only when kqueue/kevent is the selected equivalent of the mandatory central poll mechanism.

5.7 kevent

KIND	Register and receive kqueue events	PLATFORM	BSD and macOS
HEADERS	#include <sys/types.h> #include <sys/event.h> #include <sys/time.h>		

PROTOTYPE

```
int kevent(int kq, const struct kevent *changelist,  
          int nchanges, struct kevent *eventlist,  
          int nevents, const struct timespec *timeout);  
  
EV_SET(&change, ident, filter, flags, fflags, data, udata);
```

WHAT IT DOES

Applies event registrations to a kqueue and optionally waits for triggered events in the same call.

PARAMETERS

Parameter	Meaning
kq	Descriptor returned by kqueue.
changelist / nchanges	Array of registration changes and its element count; may be NULL/0.
eventlist / nevents	Output array and capacity for triggered events; may be NULL/0 for change-only calls.
timeout	Maximum wait as timespec; NULL blocks indefinitely and zero polls immediately.

RETURN VALUE

Number of events placed in eventlist; 0 when the timeout expires; -1 on call failure. A returned EV_ERROR event reports a per-change error in its data field.

ROLE IN WEBSERV

Registers EVFILT_READ or EVFILT_WRITE and dispatches returned events from one BSD/macOS server loop.

C++98 USAGE PATTERN

```
struct kevent change;  
EV_SET(&change, client, EVFILT_READ, EV_ADD | EV_ENABLE,  
      0, 0, NULL);  
if (kevent(queue_fd, &change, 1, NULL, 0, NULL) == -1)  
    /* registration failed */;
```

COMMON MISTAKES

- Check returned flags for EV_ERROR and EV_EOF.
- A changelist is applied before the call returns triggered events.
- Enable EVFILT_WRITE only while output is pending to avoid a busy loop.
- The ident field is commonly the descriptor, but interpret every record according to its filter.

SUBJECT RULE

All socket and pipe readiness must be represented in the same kqueue if this is your selected equivalent. Returned filters authorize the corresponding I/O direction.

6. Descriptor control

Put every relevant descriptor into the subject-approved nonblocking mode before the event loop depends on readiness.

6.1 fcntl

KIND	Descriptor control	PLATFORM	POSIX
HEADERS	#include <fcntl.h>		

PROTOTYPE

```
int fcntl(int fd, int cmd, ... /* arg */);
```

WHAT IT DOES

Performs command-specific operations on an open descriptor, including changing descriptor or file-status flags.

PARAMETERS

Parameter	Meaning
fd	Open descriptor to control.
cmd	Operation selector. Webserv restricts which commands/forms may be used.
arg	Optional command-specific value; its type and meaning depend on cmd.

RETURN VALUE

Command-specific nonnegative result on success; -1 on failure.

ROLE IN WEBSERV

Sets listening sockets, accepted clients, and relevant CGI pipe ends to nonblocking mode before the central readiness loop operates on them.

C++98 USAGE PATTERN

```
if (fcntl(fd, F_SETFL, O_NONBLOCK) == -1)
    /* descriptor setup failed */;
```

COMMON MISTAKES

- fcntl is variadic: the required third argument depends on cmd.
- Accepted sockets do not portably inherit every listener status flag; configure each accepted descriptor.
- General Unix examples often use F_GETFL before F_SETFL, but the Webserv subject explicitly restricts fcntl usage; do not import unlisted commands blindly.
- FD_CLOEXEC is a descriptor flag, while O_NONBLOCK is a file-status flag; follow the exact command forms accepted by your subject and evaluator.

SUBJECT RULE

The subject permits fcntl only in the stated forms involving F_SETFL, O_NONBLOCK, and FD_CLOEXEC; other flags are forbidden. The explicit nonblocking form is fcntl(fd, F_SETFL, O_NONBLOCK). Because FD_CLOEXEC normally pairs with a command such as F_SETFD, confirm the evaluator's exact interpretation before adding any other command.

Reference spine

Start with the Webserv subject for authorization and project-specific restrictions. For exact ABI details, feature-test requirements, constants, and platform differences, use the manual installed on the machine where the project is compiled and evaluated.

Linux system calls

[Linux man-pages: section 2 index](#)

https://man7.org/linux/man-pages/dir_section_2.html

C library and POSIX interfaces

[Linux man-pages: section 3 index](#)

https://man7.org/linux/man-pages/dir_section_3.html

BSD/macOS event queue

[FreeBSD kqueue\(2\) and kevent\(2\)](#)

<https://man.freebsd.org/cgi/man.cgi?query=kqueue&sektion=2>

FAST LOOKUP

Run `man 2 NAME` for system calls and `man 3 NAME` for library interfaces. If both sections exist, read the synopsis and return-value sections before coding, then read errors and notes before debugging.